ECSE 552 Final Project Proposal: CARLA - Catan Advanced Reinforcement Learning Agent

Background

Settlers of Catan is a four-player board game in which players gather resources from dice rolls, build on a randomly generated hexagonal board, and trade with the bank and each other to reach 10 victory points first. It is stochastic, partially observable (hidden hands and development cards), mixes competition with cooperation (trading), and has a long horizon and a huge branching factor.

Application. We are building a deep reinforcement learning (RL) agent that plays the full game, including player-to-player trading. Comparable games have been mastered with search plus learned evaluation [1] and large-scale self-play PPO [2]. Catan's mix of hidden information, chance and negotiation mirrors real-world problems such as multi-party negotiation, resource allocation under uncertainty, and automated trading.

Problem formulation. We model Catan as a partially observable stochastic game: the agent sees its own hand and public information, selects one legal action per decision, and must maximise its win probability against fixed opponents. Two gaps motivate a solution. First, no existing learning agent masters the full game with player trading; prior work drops players, drops trading, or hand-codes most decisions [3, 4]. Second, the win/loss reward is sparse: games last hundreds of decisions, so early placements and trades are hard to credit and learning is sample-inefficient. Rewarding one's own points is a poor fix, because a move, especially a trade, is only good relative to what it gives opponents. Better reward design would make the full game learnable on modest compute.

Aims/Goals

Design constraints

1. **Legality:** every sampled action is legal, which requires per-head masks conditioned on earlier heads.
2. **Fair information:** the policy sees only its seat's information; opponents' true hands may appear only in training losses.
3. **Symmetry:** invariance to board rotation/reflection and seat order, hence a coordinate-free graph encoder and relative seats.
4. **Reward alignment:** shaping must be potential-based to preserve optimal policies [5], must not reward proxies (trade spam, hoarding), and must measure advantage *relative* to opponents.
5. **Robustness to chance:** dice and card draws dominate single games, so results need many games, rotated seats and fixed seeds.
6. **Performance:** $\geq 99\%$ win rate against random bots and a statistically significant margin over the heuristic bot (above 25% in four-player games). A 95% rate against a competent heuristic bot is unlikely, since dice luck lets a decent amateur player win a sizeable share of games against any opponent.

Scope. We test whether opponent-aware reward shaping improves learning and final strength, focusing on trading and long-horizon credit assignment. We compare (i) sparse win/loss, (ii) own-points shaping, and (iii) opponent-aware shaping whose potential is the agent's point margin over its strongest opponent. A secondary aim is the encoder and belief ablation below. The project's scope is the full base game with maritime trading and switchable domestic trading. Out of scope: expansions, natural-language negotiation and human evaluation.

Methodology

Proposed solution. CARLA plays the full four-player game, including maritime and domestic trading, and is trained with PPO [6] over a factored action space. An action-type head comes first, then type-specific heads (intersection, edge, tile, victim, development card, resources) and a trade decoder. Each head is masked given earlier heads, so illegal actions are impossible. Unlike SettlersRL [7], the board (19 tiles, 54 intersections, 72 edges) is a coordinate-free graph processed by a GNN [8], making the policy invariant to board symmetries. Opponents, development cards and trade offers are encoded with self-attention [9]. A GRU belief module summarises public events and gets an auxiliary loss on opponents' true hands, which the simulator provides during training.

Implementation plan. Our implementation plan features (1) Engine: a from-scratch, seeded Python simulator of the full rules with legal-action generation and exact replay, already complete and tested. (2) Learning: CPU workers run self-play against a pool of past snapshots, random and heuristic bots; a GPU learner performs PPO with GAE, after a behavioural-cloning warm start on heuristic games. The three reward formulations are interchangeable config options,

trained with otherwise identical settings. (3) Architecture: MLP baseline (implemented), GNN, and GNN with attention encoders, plus a rule-based or learned belief, selected by configuration for controlled ablations.

Validation, testing, and evaluation. Engine tests check rule invariants and replay hashes over randomized games; a leakage test confirms observations ignore opponents' hidden state. Agents play tournaments against fixed bots; we report win rates with 95% confidence intervals over at least three seeds [10] (about ±3 points at 1,000 games). The main experiments are the reward comparison and an ablation over encoder, belief and domestic trading. We also track trade acceptance and rotation invariance.

Alternative solutions. Alternative solutions to playing Catan have been explored. Heuristic or search-based players [11, 12] depend on hand-engineered knowledge. Earlier learning approaches simplify the problem: [3] restrict play to two players without trading, and [4] learn only trading while a heuristic makes every other decision. The closest prior work, SettlersRL [7], has several limitations: it uses a flat tile encoder, does not mask infeasible trades, uses no demonstrations and is evaluated mostly against its own checkpoints. CARLA addresses each: masked, feasibility-aware trades; behavioural-cloning warm starts; a symmetry-respecting graph encoder, which should improve sample efficiency on our compute budget; and evaluation against fixed baselines with confidence intervals.

Feasibility and resources

No external dataset is needed: the simulator generates unlimited games, heuristic games provide demonstrations, and held-out seeds form the test set. The codebase already has the engine (about 200 tests), random and heuristic bots, environments, rewards, rollout collection, the MLP baseline, tournaments with confidence intervals and a GUI; domestic trading is under review. Still to build: PPO updates, the opponent pool, opponent-aware rewards, the GNN, attention and belief modules, and behavioural cloning. A Ryzen 9800X3D workstation with an RTX 5080 runs about 560 games/s on 8 cores, and a GPU Slurm cluster is available for seed sweeps.

Risk analysis

The main risk is that learning stalls under the sparse reward or that self-play cycles between strategies; we mitigate this with the behavioural-cloning warm start, reward shaping, and fixed bots kept in the opponent pool. Shaping itself may be exploited, for example by chasing points instead of winning, so it is potential-based and always compared against a sparse-reward baseline. Trading may degenerate into spam or disappear entirely, which feasibility masks, per-turn offer limits and trade metrics will reveal. If compute proves insufficient, we prioritise the MLP reward study and shrink the ablation grid after pilot runs. Finally, engine bugs or information leakage could silently corrupt results; invariant, replay and leakage tests run in CI, and games can be inspected visually in the GUI.

Role of group members

All four members contribute to every part of the codebase rather than owning fixed sections. Work is organised on GitHub: each task (a feature, experiment or fix) is developed on its own branch and merged through a pull request that another member reviews and that must pass continuous integration (tests, linting and type checks). We meet online at least once a week to track progress and discuss important decisions. The final report will be written the same way, with sections drafted and reviewed collaboratively instead of being assigned to individuals.

References

[1] David Silver et al. "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587 (2016), pp. 484–489. DOI: 10.1038/nature16961.

[2] Christopher Berner et al. "Dota 2 with Large Scale Deep Reinforcement Learning". In: *arXiv preprint arXiv:1912.06680* (2019). URL: https://arxiv.org/abs/1912.06680.

[3] Quentin Gendre and Tomoyuki Kaneko. "Playing Catan with Cross-Dimensional Neural Network". In: *Neural Information Processing*. Vol. 12533. Lecture Notes in Computer Science. Springer, 2020, pp. 580–592. DOI: 10.1007/978-3-030-63833-7_49.

[4] Konstantia Xenou, Georgios Chalkiadakis, and Stergos Afantenos. "Deep Reinforcement Learning in Strategic Board Game Environments". In: *Multi-Agent Systems*. Ed. by Marija Slavkovik. Springer International Publishing, 2019, pp. 233–248. DOI: 10.1007/978-3-030-14174-5_16.

[5] Andrew Y. Ng, Daishi Harada, and Stuart Russell. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping". In: *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*. Morgan Kaufmann, 1999, pp. 278–287.

[6] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017). URL: https://arxiv.org/abs/1707.06347.

[7] Henry Charlesworth. *Learning to Play Settlers of Catan with Deep Reinforcement Learning*. URL: https://settlers-rl.github.io/ (visited on 10/05/2026).

[8] William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2020. URL: https://www.cs.mcgill.ca/~wlh/grl_book/.

[9] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[10] Peter Henderson et al. "Deep Reinforcement Learning That Matters". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018. DOI: 10.1609/aaai.v32i1.11694.

[11] Michael Pfeiffer. "Reinforcement Learning of Strategies for Settlers of Catan". In: *International Conference on Computer Games: Artificial Intelligence, Design and Education*. University of Wolverhampton, 2004, pp. 384–388.

[12] István Szita, Guillaume Chaslot, and Pieter Spronck. "Monte-Carlo Tree Search in Settlers of Catan". In: *Advances in Computer Games*. Ed. by H. Jaap van den Herik and Pieter Spronck. Springer, 2010, pp. 21–32. DOI: 10.1007/978-3-642-12993-3_3.